Real-Time PPE Safety Detection & Compliance System

Complete AI Engineering Mentor Guide

A Step-by-Step Roadmap from Dataset to
Production-Ready Computer Vision Product

Built on the project's original README specification
Modular Monolith Architecture · YOLOv8 · Supervision · ByteTrack · Streamlit

Computer Vision | AI Engineering | Software Architecture | Mentor-Style

Table of Contents

0. How to Use This Guide

1. The Big Picture — System Architecture

2. Development Strategy — MVP First

Phase 1 — Environment & Tooling Setup

Phase 2 — Dataset Acquisition & Annotation Strategy

Phase 2b — Dataset Exploration & Data Quality (EDA)

Phase 3 — YOLO Training

Phase 3b — Model Evaluation

Phase 4a — The Detector Module

Phase 4b — Tracking / ByteTrack

Phase 4c — PPE Association & Violation Logic (Core)

Phase 4d — Compliance State Model

Phase 5 — Frame Processing & HUD

Phase 5b — Video Processor

Phase 6 — Detection Service (Orchestration)

Phase 7 — Streamlit Dashboard

Phase 8 — CLI Entrypoint

Phase 9 — Testing

Phase 10 — Dockerization & Deployment

Production Engineering

Performance Optimization

Advanced Product Features

Learning Roadmap

Debugging Playbook

Git Strategy

Definition of Done

Suggested Development Timeline

Final Portfolio / Product Standard

Final Demo Scenario

Final Architecture — Layered View

Engineering Rules to Live By

0. How to Use This Guide

> **MENTOR NOTE**
>
> This document is not a reference manual you skim once. It is a working roadmap. Each phase is self-contained: it tells you what to build, why, what to learn first, what files to create, how to test, what commonly goes wrong, and how you know you're ready to move on. Work through it top to bottom. Do not skip to the dashboard before the pipeline works. Do not skip testing because "it looks fine." The order of phases in this guide is deliberate and mirrors how real computer vision products get built.

Every phase in this guide follows the same internal structure so you always know where you are:

Section	Purpose
Before Starting	What must already be true / working before this phase makes sense
What We're Building & Why	The engineering purpose of this phase, in plain language
Learn First	Concepts to understand before writing code
Build	What to implement, with reasoning, not full code
Files	Exact files/modules this phase touches
Test	How to validate the phase actually works
Expected Result	What "done" looks like concretely
Common Problems	What typically breaks and why
Checkpoint	The explicit go/no-go gate before moving on

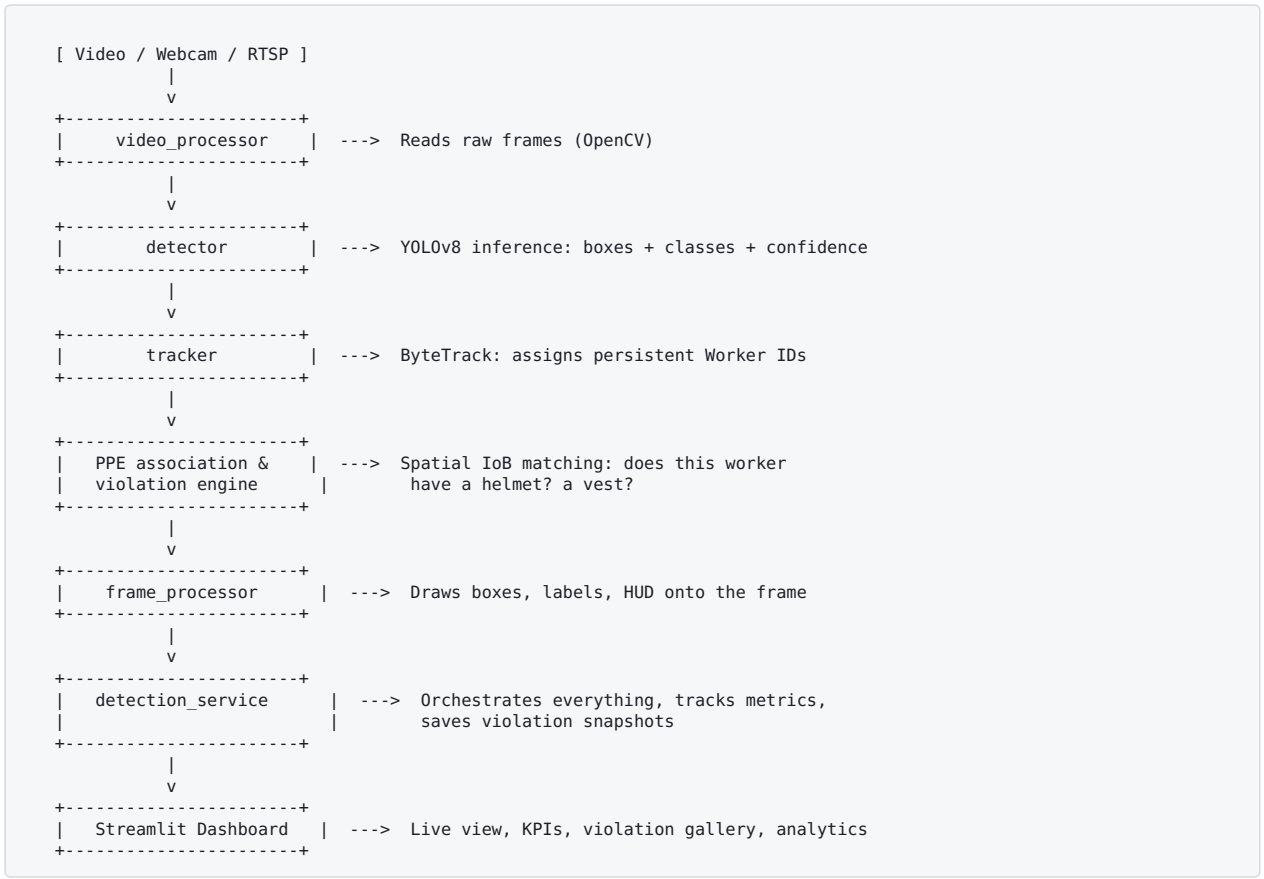
1. The Big Picture — System Architecture

Read this before writing a single line of code.

What You're Building

A real-time computer vision pipeline that watches a video feed (a file, a webcam, or an RTSP camera stream), finds every person and every piece of PPE (hard-hats, vests) in each frame, figures out *which PPE belongs to which person*, tracks each person across frames so they have a stable identity, and decides whether that person is compliant with safety rules. The result is displayed live in a dashboard and logged for later review.

The Pipeline, Component by Component



What flows between each component

From → To	Data passed
video_processor → detector	A single raw frame (NumPy array, BGR, HxWx3)
detector → tracker	`sv.Detections` — boxes, class ids, confidences, no identity yet
tracker → violation engine	`sv.Detections` with a persistent `tracker_id` attached to each person box
violation engine → frame_processor	A list of structured worker compliance states (see Phase 4d)
frame_processor → detection_service	An annotated frame (NumPy array) ready to display or write to disk
detection_service → dashboard	Annotated frame + aggregate statistics (counts, compliance %, violation events)

Why a Clean Modular Monolith (and not Microservices)

A modular monolith means: one deployable application, but internally split into strict layers with clear responsibilities and one-directional dependencies (detection knows nothing about Streamlit; the dashboard knows

nothing about YOLO internals). Compare that to microservices, where each component would be its own deployed service communicating over a network.

> **MENTOR RECOMMENDATION**
>
> For this project, microservices would be over-engineering. You have one video stream, one inference model, and one team (you). Splitting into network-separated services would add deployment complexity, network latency between every pipeline stage, and operational overhead (service discovery, health checks, distributed logging) with zero benefit at this scale. A modular monolith gives you the same separation of concerns and testability, without the distributed-systems tax. You would only reconsider this if you needed independent horizontal scaling of, say, the detector across many GPUs serving many camera feeds — a genuinely different product.

Layer Responsibilities at a Glance

Layer	Folder	Responsibility	Must NOT contain
Core	`src/core/`	Config, constants, logging setup	Any CV or business logic
Detection (Domain)	`src/detection/`	Model inference, tracking, PPE association rules	Drawing, Streamlit, I/O concerns
Processing	`src/processing/`	Frame annotation, video I/O	Decision-making about compliance
Services	`src/services/`	Orchestration of the above, metrics, snapshot saving	Low-level CV math, UI rendering
Dashboard	`src/dashboard/`	Presentation only — calls the service layer	Any detection or tracking logic

2. Development Strategy — MVP First

The order in which you build matters more than the order in which the README lists files.

COMMON MISTAKE

Starting with the Streamlit dashboard, Docker, RTSP support, multi-camera switching, or alerting before the core detection-to-compliance pipeline works on a single static image. This is the single most common reason student CV projects stall: you end up debugging three unstable layers (UI, video I/O, and model) at once instead of one at a time.

Build in this order

```
Stage A.1        Image -> YOLO -> PPE Detection -> Result
                  (Prove the model can find PPE at all)

Stage A.2        Video -> YOLO -> Person Detection -> Tracking
                  (Prove you can maintain identity across frames)

Stage A.3        Person + PPE -> Worker Compliance
                  (Prove the association/business logic works)

Stage A.4        Full Pipeline -> Dashboard
                  (Only now wrap it in a UI)
```

Each stage should produce something you can visually verify — a saved image with boxes drawn on it, a printed list of track IDs, a printed compliance dictionary — before you add the next layer. This is incremental development applied to computer vision specifically: at every stage you have one new variable to debug, not five.

CHECKPOINT — BEFORE WRITING ANY STREAMLIT CODE

You should be able to run a plain Python script that takes a video file, prints a per-frame list of `{worker_id, has_helmet, has_vest, status}` to your terminal, and saves at least one annotated frame to disk. If you cannot do this reliably, the dashboard will only make debugging harder, not easier.

Phase 1 — Environment & Tooling Setup

Goal: a clean, reproducible environment so bugs come from your code, not your setup.

Before Starting

Python 3.10+ installed on your machine, and a code editor. Nothing else required.

What We're Building & Why

Before any detection code exists, you need an isolated dependency environment, a centralized configuration object, and structured logging. Skipping this feels like it saves time, but it costs far more later: hardcoded paths and magic numbers scattered across files become impossible to tune, and unhelpful `print()` debugging becomes painful once the pipeline has five stages running per frame.

Learn First

Concept	Why you need it here
Virtual environments	Isolate this project's dependencies from your system Python and other projects
Pydantic `BaseModel` / `BaseSettings`	Type-validated, centralized configuration instead of scattered constants
Structured logging (loguru)	You will need timestamped, leveled logs once the pipeline has multiple stages running per frame
CUDA vs CPU inference	Determines your training speed and which PyTorch build you install

Build

1. Virtual environment

```
python -m venv venv
# Linux/macOS
source venv/bin/activate
# Windows
venv\Scripts\activate
```

2. Dependencies

Group them mentally so you understand why each exists, not just that it's in `requirements.txt`:

Group	Packages	Purpose
Core CV/DL	`ultralytics`, `torch`, `torchvision`, `opencv-python`, `numpy`, `pillow`	Model, tensors, image/frame handling
Tracking & annotation	`supervision`	ByteTrack wrapper, box/label drawing utilities, standardized `Detections` object
Presentation	`streamlit`	The dashboard, later
Config & utils	`pydantic`, `pyyaml`, `loguru`, `tqdm`	Settings validation, YAML parsing, logging, progress bars
Testing	`pytest`	Unit and integration tests

> ***THE README DOES NOT SPECIFY THIS***
>
> *Exact version pins. Pin at minimum `ultralytics`, `torch`, and `supervision` to specific versions once training works — Ultralytics and Supervision APIs have changed between minor versions, and an unpinned upgrade mid-project is a classic source of "it worked yesterday" bugs.*

3. Configuration module — `src/core/config.py`

Define one `Settings` class holding every threshold and path the system needs: model path, confidence threshold, IoU threshold, IoB threshold for PPE association, required PPE classes, output directories. This is the single place

you tune the system from.

```
# Skeleton only — implement the fields yourself
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    model_path: str = "models/best.pt"
    conf_threshold: float = 0.4
    iou_threshold: float = 0.5
    iob_association_threshold: float = 0.5
    # ... add the rest: required PPE classes, output dirs, device, etc.

settings = Settings()
```

4. Logger — `src/core/logger.py`

Configure loguru once, centrally, with rotation, and import the same logger everywhere instead of re-configuring per file.

Files touched this phase

```
ppe_detection/
├── venv/
├── src/core/config.py
├── src/core/logger.py
├── requirements.txt
└── .gitignore
```

Test

SMOKE TEST

Write a 5-line script that imports `torch`, prints `torch.cuda.is_available()`, imports `ultralytics` and `supervision`, and prints their versions. If this runs without ImportError, your environment is sound.

Common Problems

Symptom	Likely cause
`torch.cuda.is_available()` is `False` on a GPU machine	Installed the CPU-only PyTorch wheel instead of the CUDA build
`ImportError` for OpenCV on Linux servers	Missing system libs (`libgl1`) — common in Docker, covered in Phase 10
Pydantic import errors	Pydantic v2 split `BaseSettings` into `pydantic-settings` — install that package separately

Checkpoint — Move to Phase 2 when...

- ☐ Virtual environment activates without error
- ☐ All core packages import successfully
- ☐ `Settings()` instantiates and prints sane defaults
- ☐ A test log line appears in both console and `logs/app.log`

Phase 2 — Dataset Acquisition & Annotation Strategy

Goal: high-quality, correctly-labeled data. This determines your model's ceiling more than any training trick.

What We're Building & Why

A YOLO-format dataset split into train/val/test, covering the classes the README specifies, with enough diversity in lighting, distance, and occlusion to generalize beyond your training footage.

Learn First

- What makes a dataset "good" for object detection: class balance, annotation consistency, scene diversity
- YOLO annotation format: one `.txt` per image, each line `class_id x_center y_center width height`, normalized 0–1
- Train/val/test split rationale — why you never tune on your test set
- Dataset leakage — near-duplicate frames from the same video ending up in both train and val, inflating your validation score

Build

Sourcing data

Use a public PPE / construction-site-safety dataset from Roboflow Universe or Kaggle as your starting point, per the README. Check its license before using it in a portfolio piece — some Roboflow datasets are CC BY 4.0 (attribution required), others are more restrictive.

The key engineering decision: explicit "no-PPE" classes vs. derived logic

The README's `data.yaml` lists `hard-hat`, `safety-vest`, and also implies `no-hard-hat` / `no-safety-vest` as classes. You have two real options:

Approach	How it works	Pros	Cons
A. Explicit negative classes `no-hard-hat`, `no-safety-vest` as detector outputs	The model directly predicts "bare head" or "bare torso" as its own object class	Model learns visual cues of absence (bare head shape, skin/hair texture) directly; no dependency on your association logic being correct	Requires the dataset to have these annotated consistently; doubles the classes the model must discriminate; ambiguous when a head is simply not visible (occluded, off-frame) vs. genuinely bare
B. Derived via business logic Only detect `person`, `hard-hat`, `safety-vest`; absence = no matching PPE box found in the associated region	Your violation engine infers "no helmet" when no `hard-hat` box overlaps a person's head region above the IoB threshold	Fewer classes for the detector to learn (easier training, better per-class accuracy); one clean signal — "did we find it or not"; easier to reason about and debug	Depends entirely on your association logic and thresholds being correct; a missed detection (false negative) looks identical to a genuine violation

MENTOR RECOMMENDATION

Start with **Approach B** (derived logic) for your MVP. It is simpler to reason about, trains a smaller, more accurate detector, and — critically — it's where the interesting engineering of this project lives (Phase 4c). If your dataset already ships with explicit `no-hard-hat` annotations and you have time later, you can add Approach A as a secondary signal that cross-checks your derived logic, but don't start there.

Data config — `data/ppe_data.yaml`

```
path: ./data
train: train/images
val: val/images
names:
  0: person
  1: hard-hat
  2: safety-vest
  3: no-hard-hat
  4: no-safety-vest
  5: machinery
```

If you adopt Approach B, you can drop classes 3 and 4 from the detector entirely and derive them in code instead. Decide this before you start annotating or filtering the dataset.

Files touched this phase

```
data/
├── raw/
├── processed/
│   ├── train/images  train/labels
│   ├── val/images    val/labels
│   └── test/images   test/labels
└── ppe_data.yaml
```

Test

Spot-check 10–15 random images with their label files overlaid (draw the boxes yourself with OpenCV) to confirm annotations are aligned, not off-by-one-class or mirrored.

Common Problems

Symptom	Likely cause
Boxes drawn in the wrong place when you visualize labels	Coordinates not normalized, or width/height swapped with x/y center
Class distribution wildly skewed toward `person`	Expected — persons appear once per worker, PPE items must overlap them; consider class-weighted loss later if it hurts PPE recall

Checkpoint — Move to Phase 2b when...

- ☐ You have a clear train/val/test split with no shared source frames across splits
- ☐ You have decided Approach A vs. B and written that decision down
- ☐ Manually inspected labels look correct on at least 10 sample images

Phase 2b — Dataset Exploration & Data Quality (EDA)

Goal: know your data's weaknesses before you blame the model for them.

COMMON MISTAKE

Jumping straight to training and only inspecting data quality after getting a disappointing mAP. Every hour spent in EDA saves several hours of confused hyperparameter tuning later.

What to inspect, and why each matters

Check	Why it matters
Number of images / annotations per split	Too few images per class and the model won't generalize
Class distribution	Severe imbalance (e.g. 10x more `person` than `safety-vest`) biases the model toward the majority class
Bounding box sizes & aspect ratios	Tells you if PPE items are consistently tiny (helmets from a distance) — small objects are harder for YOLO to detect and may need a higher input resolution
Missing labels	Images with zero annotations silently teach the model "nothing is here" — verify these are truly empty scenes, not annotation errors
Corrupt images	Will crash training mid-epoch if not filtered first
Duplicate / near-duplicate images	Inflates your effective dataset size and can leak across train/val
Occlusion & difficult images	Understand the model's realistic ceiling — heavily occluded workers will always be harder

Build

Create a single exploration notebook. Do not write full analysis code here — define the objectives, then implement them yourself:

```
notebooks/
└── 01_data_exploration.ipynb
```

The notebook's objectives, in order:

1. Load and count images/labels per split; print a class-frequency table
2. Plot a histogram of bounding box width/height (normalized) to spot a small-object problem early
3. Scan for zero-byte or unreadable images and quarantine them
4. Overlay labels on a random grid of 12–16 images and visually inspect for annotation errors
5. Compute a simple perceptual hash or file-hash based duplicate check across splits

Test

The notebook's output itself is the test — you should end this phase with a written list of your dataset's known weaknesses (e.g. "vest annotations are inconsistent when the vest color matches the background", "few night-time images").

Checkpoint — Move to Phase 3 when...

- ☐ You have a class distribution table and know which classes are under-represented
- ☐ Corrupt/duplicate images have been removed
- ☐ You have written down 2–3 specific known weaknesses to watch for during evaluation

Phase 3 — YOLO Training

Goal: a trained detector that finds people and PPE reliably enough to build on.

Before Starting

Phase 2b complete — you know your dataset's class balance and weaknesses.

Learn First — Object Detection Fundamentals

Concept	Plain explanation
Bounding box	A rectangle (x, y, w, h) locating an object in the image
Class	What the object is (person, hard-hat, ...)
Confidence	The model's own certainty score for a prediction, 0–1
IoU (Intersection over Union)	Overlap between two boxes divided by their combined area — measures how well a predicted box matches a ground-truth box
NMS (Non-Max Suppression)	Removes duplicate overlapping predictions for the same object, keeping the highest-confidence one
Precision	Of everything the model flagged, what fraction was actually correct
Recall	Of everything that was actually there, what fraction did the model find
mAP@50	Mean average precision at an IoU threshold of 0.5 — a lenient, commonly-reported metric
mAP@50-95	Averaged across IoU thresholds 0.5 to 0.95 — a stricter, more localization-sensitive metric
Confusion matrix	Shows which classes get mistaken for which (e.g. baseball cap predicted as hard-hat)

Choosing a Model Size

Model	Relative speed	Relative accuracy	When to choose
YOLOv8n (nano)	Fastest	Lowest	Edge devices (Jetson, low-power CPU inference), or rapid early iteration
YOLOv8s (small)	Fast	Good balance	**Default choice for this project** — real-time on a modest GPU with solid accuracy
YOLOv8m (medium)	Slower	Higher	If your dataset is large and diverse enough to benefit, and you have GPU headroom

Decide based on: your GPU's VRAM (larger models need more), your target FPS, your dataset size (a bigger model on a small dataset overfits faster), and your eventual deployment target (edge devices favor smaller models).

Build

Baseline training run — `training/train.py`

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")
results = model.train(
    data="data/ppe_data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    device=0,   # GPU index, or "cpu"
    project="models/runs",
    name="ppe_yolov8s",
)
```

Understand each argument before changing it:

Argument	What it controls	Tuning notes
`epochs`	Full passes over the training set	Start at 50; watch for validation loss plateauing/rising (overfitting) via early stopping
`imgsz`	Input resolution	640 is standard; raise it if PPE items are small and consistently missed
`batch`	Images per gradient step	Reduce if you hit CUDA out-of-memory errors
`device`	GPU index or CPU	CPU training is viable only for very small experiments
Augmentation (defaults)	Random flips, color jitter, mosaic	Ultralytics' defaults are reasonable; don't hand-tune this before you have a baseline

> **MENTOR NOTE**
> Run the baseline with defaults first. Do not tune augmentation, learning rate, or architecture before you have a number to improve on. Your first goal is simply: does training converge, and is the model better than random?

Experiment tracking

Ultralytics writes results (metrics, plots, weights) to `project/name` automatically per run. Keep the benchmark table below updated by hand as you try variants — this is your experiment log. Evaluation Resulats

Model	Epochs	Image Size	Batch	Precision	Recall	mAP50	mAP50-95	Speed
yolov8s (baseline)	50	640	16	~~—~~0.9533	~~0~~.7979	~~—~~0.9035	~~—~~ 0.5642	~~—~~4.9ms

Files touched this phase

```
training/train.py
training/evaluate.py
models/runs/ppe_yolov8s/     (Ultralytics output: weights, plots, logs)
models/best.pt               (copied from runs/.../weights/best.pt)
```

Common Problems

Symptom	Likely cause	What to try
CUDA out of memory	Batch size or image size too large for your GPU	Halve the batch size before reducing image size
Loss doesn't decrease	Learning rate issue, corrupted labels, or wrong `data.yaml` paths	Verify a handful of batches visually using Ultralytics' training batch preview images
Training loss low, validation loss high	Overfitting — likely too few images or too many epochs for dataset size	More data, augmentation, or early stopping

Checkpoint — Move to Phase 3b when...

- ☐ Training completes without crashing and produces `best.pt`
- ☐ Training loss trended downward across epochs
- ☐ You have a filled-in row in the benchmark table above

Phase 3b — Model Evaluation

Goal: know whether the model is actually good enough to build the rest of the system on, before you invest weeks on top of it.

Why Accuracy Alone Is Not Enough

A single mAP number hides a lot. A model can have decent mAP50 while systematically failing on exactly the cases your product needs — small distant helmets, or confusing a baseball cap with a hard-hat. You need to look past the summary metric.

What to check

Failure mode	What it looks like	Why it matters for this product
False positives	Model detects PPE that isn't there	Falsely marks a non-compliant worker as compliant — a safety-critical false negative for the business
False negatives	Model misses real PPE	Falsely flags a compliant worker as violating — erodes trust in the system
Class confusion	Confusion matrix shows hard-hat ↔ head confusion, or vest ↔ background confusion	Directly determines whether your association logic gets reliable inputs
Small-object misses	Distant workers' helmets never detected	Real construction cameras are often wide-angle; this is a realistic deployment condition, not an edge case
Occlusion failures	Partially hidden workers missed entirely	Common on busy sites; decide if this is acceptable for your MVP
Lighting sensitivity	Performance drops in low light or backlit scenes	Test explicitly if your target deployment includes dawn/dusk/indoor lighting

Practical Evaluation Checklist

1. Run `training/evaluate.py` against the validation set and record precision, recall, mAP50, mAP50-95 per class (not just overall)
2. Open the confusion matrix plot and identify the two most-confused class pairs
3. Run inference on 20–30 validation images and manually eyeball the predicted boxes against your own judgment
4. Deliberately test on a few images with distant/small workers and a few with partial occlusion
5. Compare training-set performance to validation-set performance — a large gap signals overfitting

If Something Is Wrong

Problem	What to do
mAP is low overall	Return to EDA — check for label errors before touching hyperparameters
Recall is low	Model is missing real objects — check for under-represented classes or too-small objects; consider higher `imgsz`
Precision is low	Model is hallucinating objects — check for label noise or class visual similarity (e.g. yellow rain jackets vs. safety vests)
Helmet detection specifically poor	Check helmet box sizes in EDA — helmets are small relative to frame; verify diverse helmet colors are represented
Vest detection specifically poor	Check for vest-color/background-color similarity in your training images
Person detection poor	Unusual — check for occlusion-heavy training images or annotation errors on the person class
Validation much worse than training	Overfitting — add augmentation, more data, or reduce epochs/model size

CHECKPOINT — MOVE TO PHASE 4A WHEN...

You have per-class metrics recorded, you've manually reviewed at least 20 predictions, and you can state in one sentence what your model is currently bad at. "Good enough for the MVP, weak on distant helmets" is a valid, useful checkpoint answer — perfection is not required to proceed.

Phase 4a — The Detector Module

Goal: a clean, reusable wrapper around YOLO inference that the rest of the system depends on.

Before Starting

You have `models/best.pt` from Phase 3, evaluated in Phase 3b.

What We're Building & Why

`src/detection/detector.py` should be the *only* place in the codebase that knows how to call the YOLO model. Everything downstream (tracker, violation engine, frame processor) should depend only on a standardized output format, never on Ultralytics' raw result objects directly. This is the Dependency Inversion principle in practice: swap YOLO for a different detector later, and only this one file changes.

Learn First

- Why wrap a third-party library instead of calling it directly everywhere (testability, replaceability, one place to change confidence logic)
- `supervision.Detections` — a standardized container for boxes, class ids, confidences, and (later) tracker ids
- Confidence thresholding at inference time vs. at training time

Build

Class skeleton — implement the body yourself

```
import numpy as np
import supervision as sv
from ultralytics import YOLO

class PPEDetector:
    def __init__(self, model_path: str, conf_threshold: float, device: str = "cpu"):
        # Load the model once here, not per-frame.
        ...

    def detect(self, frame: np.ndarray) -> sv.Detections:
        """
        Input:  a single BGR frame as a NumPy array (H, W, 3)
        Output: sv.Detections with .xyxy, .confidence, .class_id populated
        """
        ...
```

What you need to implement

Responsibility	Notes
Constructor	Load the model **once** and store it as an instance attribute — never reload per frame, that will destroy your FPS
Device placement	Move the model to GPU if available; read this from your `Settings` object, don't hardcode it
Confidence threshold	Apply at inference call time via the Ultralytics API's own `conf` parameter
Output conversion	Convert Ultralytics' native result object into `sv.Detections` — Supervision provides a helper for this (`sv.Detections.from_ultralytics()`); read its docs rather than hand-rolling the conversion

COMMON MISTAKE

Instantiating `PPEDetector` (and therefore loading the model from disk) inside a loop or inside a function called per-frame. Model loading is expensive; do it once at startup and reuse the instance.

Files touched this phase

```
src/detection/detector.py
```

Test

1. Call `detect()` on a single sample image; print the number of detections and their classes
2. Call `detect()` on an all-black `np.zeros((640,640,3), dtype=np.uint8)` frame — should return zero detections, not crash
3. Time 20 consecutive calls on the same frame to get a rough per-frame inference latency baseline

Expected Result

A `PPEDetector` instance that, given any BGR frame, returns a populated (or empty) `sv.Detections` object in well under 100ms on GPU, or a few hundred ms on CPU with YOLOv8s.

Checkpoint — Move to Phase 4b when...

- [ ] `detect()` works on real images and returns sane boxes
- [ ] `detect()` handles an empty/black frame without crashing
- [ ] You have a rough per-frame latency number

Phase 4b — Tracking / ByteTrack

Goal: give each worker a stable identity across frames, so "Worker #3" means the same person from frame to frame.

What We're Building & Why

Detection alone re-discovers objects independently every single frame — it has no memory. If two workers walk past each other, a detector-only system has no way to know which box in frame 101 corresponds to which box in frame 100. Tracking solves this by matching detections across frames and assigning a persistent ID.

Learn First — Detection vs. Tracking

Detection	Tracking
Answers: "what objects are in this frame?"	Answers: "is this the same object I saw last frame?"
No memory between frames	Maintains state (position history, velocity estimate) per tracked object
Runs independently every frame	Predicts where each track should be, then matches new detections to predictions

Concepts to Understand

- **Worker IDs** — a stable integer assigned when a track is first created, kept until the track is lost
- **Track lifecycle** — created (new detection with no match), active (matched each frame), lost (no matching detection for N frames), removed
- **Track persistence** — most trackers tolerate a few missed frames (occlusion) before dropping a track, to avoid ID churn
- **Occlusion** — when a worker is briefly hidden behind another object or worker, a good tracker keeps the ID alive briefly rather than immediately creating a new one
- **Lost tracks** — what happens when a worker leaves frame or is occluded too long — the ID is retired
- **Re-identification limits** — ByteTrack is a motion-based tracker, not an appearance-based re-identifier; if a worker leaves and re-enters frame later, they will very likely get a *new* ID. This is a real limitation, not a bug — be ready to explain this in your final demo/interview.

ByteTrack, Conceptually

ByteTrack's key idea: most trackers throw away low-confidence detections before matching, which loses real objects that are just partially occluded or blurry. ByteTrack instead matches *high*-confidence detections to existing tracks first, then makes a second matching pass using the *low*-confidence detections against whatever tracks remain unmatched — recovering objects that would otherwise be dropped. This is why it performs well under occlusion, which matters directly for a busy construction site.

Build

```
# src/detection/tracker.py — skeleton, implement the body yourself
import supervision as sv

class WorkerTracker:
    def __init__(self):
        self._tracker = sv.ByteTrack()

    def update(self, detections: sv.Detections) -> sv.Detections:
        """
        Input:  sv.Detections for the current frame — filter to `person`
                class BEFORE calling this, tracking only makes sense for people
        Output: the same detections, now with .tracker_id populated
        """
        ...
```

> **MENTOR NOTE**
> Only feed `person` detections into the tracker. PPE items (helmets, vests) don't need persistent IDs of their own — they get associated to a tracked person in Phase 4c instead. Tracking PPE items independently would be wasted work and a source of confusing bugs.

Files touched this phase

```
src/detection/tracker.py
```

Test Plan Using a Short Video

1. Record or find a 10–15 second clip with 1–2 people walking
2. Run detector → tracker frame by frame, printing each frame's set of `tracker_id`s
3. Verify: the same person keeps the same ID across consecutive frames
4. Verify: if a person briefly leaves and re-enters frame, note whether they get a new ID (expected — see re-identification limits above)
5. Test with 2 people who cross paths — check whether IDs swap (a known ByteTrack edge case under heavy occlusion)

Common Problems

Symptom	Likely cause
IDs change every frame even for a stationary person	Feeding unfiltered detections (including PPE boxes) into the tracker, confusing the matching
ID swaps when two people cross	A known limitation of motion-based tracking under close occlusion; document it rather than trying to "fix" it in Phase 4b — appearance-based re-ID is out of scope for the MVP

Checkpoint — Move to Phase 4c when...

- [ ] A single person keeps a stable ID across a test clip
- [ ] You understand and can explain ByteTrack's two-pass matching in your own words
- [ ] You've observed and documented at least one ID-swap or ID-churn case

Phase 4c — PPE Association & Violation Logic

This is the core intellectual contribution of the whole project. Slow down here.

The Problem

You now have a set of tracked `person` boxes and a set of untracked `hard-hat` and `safety-vest` boxes, all detected independently in the same frame. Nothing tells you which helmet belongs to which person. If there are three workers in frame and two helmets, you must determine: which two workers are wearing them, and which one isn't? This is a geometry problem, not a machine-learning problem — you solve it with spatial reasoning on the boxes you already have.

Learn First — Bounding Box Geometry

Term	Meaning
Intersection	The overlapping rectangular region between two boxes (can be zero-area if they don't overlap)
IoU (Intersection over Union)	Intersection area ÷ area of the union of both boxes. Symmetric — treats both boxes equally.
IoB (Intersection over Box)	Intersection area ÷ area of *one specific box* (the PPE item, here). Asymmetric — measures "how much of this small box is inside that other box."
Containment	A special case where one box is (almost) entirely inside another

Why IoB, Not IoU, for This Problem



A helmet box is almost always much smaller than a person box. If you used plain IoU, a helmet that is 100% inside a person's head region would still score a *low* IoU, because the union (person box size) is so much larger than the intersection — IoU penalizes the size mismatch even when containment is perfect.

```
IoB =  Area(Intersection)  /  Area(PPE Bounding Box)
```

IoB instead asks a more useful question: "of the helmet's own area, how much of it sits inside the person's head region?" A helmet fully contained in the head region scores IoB ≈ 1.0 regardless of how much bigger the person box is. This matches the real question you're asking: does this PPE item belong to this specific worker?

Build — `src/detection/violation.py`

Break the module into small, independently testable functions. Do not write one giant function — each of these should be unit-testable in isolation (see Phase 9).

Function	Responsibility	Input → Output
`intersection_area(box_a, box_b)`	Pure geometry — compute overlapping rectangle area	Two (x1,y1,x2,y2) tuples → float
`calculate_iob(ppe_box, region_box)`	Apply the IoB formula above	Two boxes → float in [0,1]
`get_head_region(person_box)`	Slice the top 30% of a person box	Person box → sub-box
`get_torso_region(person_box)`	Slice the middle 50% of a person box	Person box → sub-box
`associate_ppe_to_worker(person_boxes, ppe_detections)`	For each person, find the best-matching PPE item per required region using IoB above threshold	Tracked persons + PPE detections → per-person PPE assignment
`evaluate_compliance(assignment)`	Turn a PPE assignment into the structured state from Phase 4d	Assignment → compliance dict

Function skeletons — implement the logic yourself

```
def intersection_area(box_a, box_b) -> float:
    ...

def calculate_iob(ppe_box, region_box) -> float:
    # IoB = intersection / area(ppe_box)
    ...

def get_head_region(person_box):
    # top 30% of the person box height
    ...

def get_torso_region(person_box):
    # middle 50% of the person box height
    ...

def associate_ppe_to_worker(person_detections, ppe_detections, iob_threshold: float):
    """
    For each tracked person:
      - compute head region, check overlap with hard-hat detections
      - compute torso region, check overlap with safety-vest detections
      - if best IoB > iob_threshold, assign that PPE item to this worker
    Returns: {tracker_id: {"has_helmet": bool, "has_vest": bool, ...}}
    """
    ...

def evaluate_compliance(assignment: dict) -> dict:
    # turn has_helmet/has_vest into status + missing_items, per Phase 4d
    ...
```

Edge Cases You Must Handle

Case	Correct handling
No helmet detected anywhere in frame	Worker's `has_helmet` is False; no crash from an empty PPE detection list
Helmet detected but geometrically closer to a different person	Assign to the person with the highest IoB, not just any nearby person — this is why you compute IoB against every candidate, not the first match
Two workers standing very close together	Both head regions may overlap the same helmet box; assign to the single highest-IoB match, and consider only allowing one PPE item to be claimed once (don't let one helmet satisfy two people)
Overlapping/occluding workers	Track association may already be unstable here (Phase 4b); accept degraded accuracy in this case rather than trying to force certainty
PPE detected outside any person box	Ignore it — a helmet sitting on a table isn't a worker's helmet
Partial occlusion of the PPE item itself	IoB is computed on whatever box the detector returned; if the detector under-detects a partially hidden helmet, that's a detection-quality issue (Phase 3b), not an association-logic bug
False PPE detection (model hallucination)	Out of scope for this module — mitigate at the confidence-threshold level in Phase 4a
Missing person detection (worker not detected at all)	That worker simply doesn't appear in this frame's output — cannot be evaluated for compliance if they were never detected
Multiple PPE items of the same type near one person	Take the highest-IoB one; log a warning, since this may indicate a detector duplicate needing better NMS

> **COMMON MISTAKE**
> Assigning PPE greedily in detection order instead of by best-match. If you loop through PPE detections and assign each to the "first" overlapping person you find, you can incorrectly assign a helmet to the wrong worker when two people are close together. Always compare against *all* candidate persons and pick the maximum IoB.

Test

This is the single most important module to unit test in the entire project. Build a small suite of synthetic bounding boxes (no images, no model needed) covering:

- 0% overlap → IoB should be 0

- Full containment → IoB should be ≈1.0
- Partial overlap at exactly your threshold → verify the boundary behavior (is it inclusive?)
- Two people, one helmet positioned closer to person B — verify it's assigned to B, not A

Full test matrix appears in Phase 9.

Checkpoint — Move to Phase 4d when...

- ☐ All five functions above exist and pass basic manual checks with hand-computed example boxes
- ☐ You can explain, out loud, why IoB was chosen over IoU for this specific problem
- ☐ The "two close workers, one helmet" edge case has a defined, deliberate behavior — not accidental behavior

Phase 4d — Compliance State Model

Goal: one clean, structured representation of "is this worker safe right now" that the rest of the system builds on.

What We're Building & Why

Per the README's structure, represent each tracked worker's state as a single structured object:

```
{
    "worker_id": 1,
    "has_helmet": True,
    "has_vest": False,
    "status": "VIOLATION",
    "missing_items": ["safety-vest"]
}
```

Why Structured State Beats Scattered Booleans

MENTOR RECOMMENDATION

If instead you passed around loose variables like `has_helmet_worker_1`, `violation_flag_worker_1`, tracking them across the frame processor, service layer, and dashboard would require threading many separate values everywhere and keeping them in sync. A single dict/dataclass per worker means every downstream consumer (drawing code, metrics, dashboard) reads from one object, and adding a new PPE type later (say, `has_mask`) means touching one struct definition, not every function signature in the codebase.

Consider defining this as a small dataclass or Pydantic model rather than a raw dict once you're past the prototyping stage — it gives you type checking and autocomplete for free.

How This State Flows Through the System

```
violation.py (evaluate_compliance)
        |
        v
List[WorkerComplianceState]   <-- one per tracked worker, this frame
        |
        v
frame_processor.py   -- reads status to choose box color / label text
        |
        v
detection_service.py -- aggregates into running counters, triggers
                        snapshot saving on VIOLATION status
        |
        v
dashboard/app.py     -- displays KPI cards and violation gallery
```

Possible Status Values

Status	Meaning
`COMPLIANT`	All required PPE items detected and associated
`VIOLATION`	One or more required PPE items missing
`UNKNOWN`	Not enough information to judge — see below

Is an UNKNOWN State Useful?

MENTOR RECOMMENDATION

Yes — add it. Consider a worker whose head region is out of frame, or a track that just appeared this frame with too little history. Silently defaulting such cases to `VIOLATION` creates false alarms; silently defaulting to `COMPLIANT` creates a safety blind spot that's worse. An explicit `UNKNOWN` status is honest about the system's uncertainty and lets the dashboard visually distinguish "flagged" from "cannot currently assess" — which matters a great deal if a real safety officer is trusting this system's alerts.

Files touched this phase

```
src/detection/violation.py   (extends evaluate_compliance from Phase 4c)
```

Checkpoint — Move to Phase 5 when...

- [ ] Every tracked worker produces exactly one compliance state per frame
- [ ] You've decided how `UNKNOWN` is triggered and documented it
- [ ] You can print a full frame's worth of compliance states to the console and they look correct on a test clip

Phase 5 — Frame Processing & HUD Annotation

Goal: turn compliance decisions into a clear visual — without mixing decision logic into drawing code.

What We're Building & Why

`src/processing/frame_processor.py` takes the frame plus the compliance states from Phase 4d and draws boxes, labels, and an on-screen HUD. It makes zero decisions about compliance — it only visualizes decisions already made upstream.

Decision Logic vs. Visualization Logic

> **MENTOR NOTE — AN IMPORTANT SOFTWARE ENGINEERING CONCEPT**
>
> It is tempting to compute "is this worker compliant" inline while drawing, since you already have the boxes right there. Resist this. If compliance logic lives inside the drawing function, you cannot unit-test it without a real frame and without OpenCV, and you cannot reuse it for anything other than annotated video (for example, a JSON report or a webhook alert in Phase "Advanced Product Features" would need the same decision without any drawing at all). Keep `violation.py` as the single source of truth for compliance, and let `frame_processor.py` be a pure consumer of its output.

Build

Color coding

Color	Meaning
Green	Fully compliant worker
Red	Worker with a PPE violation
Yellow	PPE item boxes themselves (hard-hat, vest)

Tools to use

- `supervision.BoxAnnotator` — draws bounding boxes, colorable per-detection
- `supervision.LabelAnnotator` — draws text labels (worker ID + status) near each box
- `supervision.TraceAnnotator` — draws a trailing path showing recent movement per tracked worker

HUD bar

Overlay a small on-screen readout: current FPS, total active workers, compliant count, violation count. This is drawn from the aggregate metrics computed downstream in the service layer (Phase 6) — the frame processor receives these numbers, it doesn't compute them.

Files touched this phase

```
src/processing/frame_processor.py
```

Test

Feed a hand-constructed list of compliance states (some COMPLIANT, some VIOLATION) against a sample frame and visually verify: correct colors, readable labels, HUD numbers match your synthetic input.

Checkpoint — Move to Phase 5b when...

- ☐ Compliant and violating workers are visually distinguishable at a glance
- ☐ HUD numbers update correctly when you change the synthetic input
- ☐ No compliance decisions are made inside this file — only rendering

Phase 5b — Video Processor

Goal: a reliable frame source, built incrementally from the simplest case to the hardest.

Build Incrementally — and Why

1. **MP4 file first.** Deterministic, repeatable, no hardware dependency — the easiest case to debug your pipeline against.
2. **Webcam second.** Introduces live, unbounded streams and real-time constraints, but still local and reliable.
3. **RTSP last.** Network streams introduce reconnection logic, variable latency, and dropped-frame handling — the hardest case. Don't attempt this until MP4 and webcam both work cleanly.

> **COMMON MISTAKE**
> Trying to develop and debug the whole pipeline against a flaky RTSP camera from day one. Network issues will masquerade as pipeline bugs and waste your time. Get everything else solid on a static MP4 file first.

Build — `src/processing/video_processor.py`

Design a generator function so the rest of the system can iterate over frames without caring about the source:

```
def get_frame_stream(source):
    """
    source: a file path, an integer webcam index (0), or an RTSP URL string
    Yields: frames one at a time (NumPy arrays)
    """
    cap = cv2.VideoCapture(source)
    ...
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        yield frame
    cap.release()
```

Concepts to Cover

Concept	Why it matters
`cv2.VideoCapture`	Unified interface for file, webcam, and RTSP sources
Frame generators	Lets consumers process one frame at a time without loading an entire video into memory
FPS & resolution	Read from the source (`cap.get(cv2.CAP_PROP_FPS)`) so output video timing matches input
`cv2.VideoWriter` & codec	Needed to save annotated output video; codec choice affects compatibility and file size
End-of-stream handling	`cap.read()` returns `False` at end of file — must break the loop cleanly, not crash
RTSP reconnection	Network streams can drop; production code should attempt to reconnect rather than terminating on the first failure

Files touched this phase

```
src/processing/video_processor.py
```

Common Problems

Symptom	Likely cause
Webcam opens but frames are always empty	Wrong device index, or another app has the camera locked
Output video won't play in some players	Codec/container mismatch — try `mp4v` for `.mp4` as a safe default
RTSP connection hangs indefinitely	No timeout set on `VideoCapture` ; add explicit timeout/retry logic

Checkpoint — Move to Phase 6 when...

- ☐ MP4 playback yields correct frames end-to-end
- ☐ Webcam streaming works and terminates cleanly on quit
- ☐ You've at least attempted RTSP against a public test stream, even if reconnection logic is basic

Phase 6 — Application Service Orchestration

Goal: one clean coordination point that ties detector, tracker, violation engine, and frame processor together.

What We're Building & Why

Up to now, each module works in isolation. `src/services/detection_service.py` is where they meet. This is your Service Layer — it owns the lifetime of the detector, tracker, and violation engine, and exposes one method the rest of the app calls: `process_frame(frame)`.

The Orchestration Sequence

```
process_frame(frame)
        |
        v
    detect()            (Phase 4a)
        |
        v
    track()              (Phase 4b)
        |
        v
    associate PPE         (Phase 4c)
        |
        v
    evaluate compliance    (Phase 4c/4d)
        |
        v
    save violation snapshot (if any worker is VIOLATION)
        |
        v
    annotate frame          (Phase 5)
        |
        v
    return frame + statistics
```

Build

Responsibility	Notes
Own component lifetimes	Construct `PPEDetector`, `WorkerTracker` once in `__init__`, reuse across calls
`process_frame(frame)`	Run the full sequence above and return (annotated_frame, stats)
Snapshot saving	When a worker's status is `VIOLATION`, save a high-resolution crop or full frame to `data/outputs/violations/` with a timestamp — but throttle this (don't save 30 snapshots/second for the same ongoing violation)
Running metrics	Track total unique workers seen (via the set of all tracker IDs observed), total violations recorded, and a rolling compliance percentage

> ***THE README DOES NOT SPECIFY THIS***
>
> *Exactly how snapshot throttling should work. A reasonable default: save a new snapshot for a given `tracker_id` only if no snapshot was saved for that same worker in the last N seconds, to avoid flooding disk with near-identical images of the same sustained violation.*

What Should NOT Be Inside This Service

- Geometry math (IoB, region slicing) — belongs in `violation.py`
- Drawing logic — belongs in `frame_processor.py`
- Streamlit widgets or UI state — belongs in the dashboard layer
- Model loading details — the service calls `PPEDetector`, it doesn't know how YOLO works internally

Files touched this phase

```
src/services/detection_service.py
```

Test

Run `process_frame()` across a full test video in a plain script (no dashboard yet) and print per-frame stats to the console. Confirm violation snapshots appear on disk only when expected, and that metrics accumulate sensibly across the whole clip.

Checkpoint — Move to Phase 7 when...

- ☐ A full video can be processed end-to-end via this service with no dashboard involved
- ☐ Snapshot files appear correctly and are throttled sensibly
- ☐ Running metrics (unique workers, violation count, compliance %) look correct against your manual expectation for the test clip

Phase 7 — Interactive Streamlit Dashboard

Goal: an intuitive UI for a safety officer — built only after the pipeline is proven solid.

GATE — DO NOT START THIS PHASE UNTIL

Phase 6's `process_frame()` works reliably against a full test video from a plain script. If it doesn't, any bug you see in the dashboard could be a UI bug, a state bug, or a pipeline bug — and you won't be able to tell which.

Build Incrementally

Version	Scope	Why this order
V1	Upload a single image, run detection, show result	Simplest possible Streamlit + service integration; validates the wiring
V2	Upload a video file, process it, show output	Introduces the video loop inside Streamlit's execution model
V3	Live processing loop with a stop control	Introduces Streamlit session state for a long-running loop
V4	Metric cards (active/compliant/violations/compliance %)	Surfaces the service layer's aggregate stats
V5	Violation snapshot gallery	Displays saved evidence images with timestamps
V6	Analytics charts (compliance over time, violation breakdown)	Historical view on top of already-working live view
V7	Webcam / RTSP source switching	Hardest source types, added last, on top of an already-working UI

MENTOR NOTE

Each version above should be a working, demoable state before you move to the next. This is the same incremental philosophy as Phase 2 (Development Strategy) applied one layer up. Resist the urge to build V1–V7 in one sitting without testing between them — Streamlit's rerun-on-interaction execution model has its own learning curve, and debugging state issues is much easier one feature at a time.

Build

Layout

- **Sidebar:** source selector (upload / webcam / RTSP), confidence slider (0.10–1.00), toggles for box display, worker trails, snapshot saving
- **Top row:** `st.metric` cards — Active Workers, Compliant Workers, Active Violations, Overall Compliance Rate
- **Main viewport:** left column = live annotated video; right column = live violation feed with timestamped snapshot cards
- **Historical section:** line chart of compliance rate over time, bar chart of most common violation types

Files touched this phase

```
src/dashboard/app.py
```

Test

Manually walk through each toggle and control after every version above. There is no meaningful way to unit test Streamlit UI itself — your test is manual interaction plus the fact that the underlying service layer is already covered by Phase 9's automated tests.

Common Problems

Symptom	Likely cause
Video processing restarts from scratch on every UI interaction	Streamlit reruns the whole script top-to-bottom on each interaction — state not properly held in `st.session_state`
UI freezes during video processing	Long-running loop blocking Streamlit's single-threaded execution — process in chunks with periodic UI updates, or use Streamlit's native video/status placeholders

Checkpoint — Move to Phase 8 when...

- ☐ V1 through at least V5 work reliably
- ☐ You can demo the full flow: upload a video, watch live annotation, see metrics update, see a violation captured in the gallery

Phase 8 — CLI Entrypoint

Goal: a developer-friendly way to run the system without opening the dashboard every time.

Learn First

`argparse` — Python's standard library tool for building command-line interfaces with subcommands, typed arguments, and auto-generated help text.

Build — `main.py`

Support three subcommands, per the README:

```
python main.py dashboard
python main.py predict-image --source input.jpg --output out.jpg
python main.py predict-video --source input.mp4 --output out.mp4
```

Command	Behavior
`dashboard`	Launches `streamlit run src/dashboard/app.py` as a subprocess, or instructs the user to run it directly
`predict-image`	Runs the detection service on a single image, saves an annotated result
`predict-video`	Runs the detection service across a full video file (or webcam index `0`), saves annotated output

Argument Validation

- Verify `--source` paths exist before attempting to open them; fail with a clear error message, not a raw stack trace
- Validate that `--output`'s parent directory exists or create it
- Handle the webcam-index case (`--source 0`) as an integer, not a file path

Files touched this phase

```
main.py
```

Test

Run each subcommand against a valid input, then again against a missing/invalid path, and confirm the error message is clear rather than a raw traceback.

Checkpoint — Move to Phase 9 when...

- ☐ All three subcommands run successfully against valid inputs
- ☐ Invalid inputs produce a clear error, not a crash

Phase 9 — Unit Testing & Robustness Verification

Goal: prove your business logic doesn't silently break, especially the geometry math from Phase 4c.

Testing as an Engineering Discipline

Test type	What it checks	Where it applies here
Unit tests	One function in isolation, no external dependencies	IoB math, region slicing, compliance evaluation
Integration tests	Multiple components working together	detector → tracker → violation engine on a real frame
Smoke tests	Does it run at all, on the simplest possible input	Detector on an empty frame
Regression tests	A previously-fixed bug doesn't come back	Add one whenever you fix a real bug in association logic
Edge cases	Boundary and unusual inputs	See the full matrix below

Testing Matrix

Geometry (unit)

- 0% overlap between two boxes → IoB is 0
- Partial overlap → IoB matches a hand-computed expected value
- Full overlap (PPE box entirely inside region) → IoB ≈ 1.0

Compliance (unit)

- Helmet + vest both present → status COMPLIANT
- Helmet only → VIOLATION, missing_items = ["safety-vest"]
- Vest only → VIOLATION, missing_items = ["hard-hat"]
- Neither → VIOLATION, missing_items = both
- Insufficient information → UNKNOWN (if you implemented Phase 4d's UNKNOWN state)

Detector (integration, real model)

- Valid image → returns sane, non-empty detections on a known test image
- Empty/black image → returns zero detections without crashing
- Wrong input shape/type → raises a clear error rather than an obscure one from deep inside Ultralytics

Tracker (integration)

- Single worker across several frames → stable ID
- Multiple workers → distinct stable IDs
- Worker enters mid-clip, another leaves → IDs assigned/retired correctly

Full pipeline (integration)

```
frame -> detector -> tracker -> violation engine -> processor
```

Run this chain against 2–3 short real clips and assert the final output shape/type is correct, without asserting exact detection values (which can shift slightly between model versions).

What to Mock vs. What to Test with Real Models

Mock (fast, deterministic)	Real model (slower, integration confidence)
Geometry functions — use hand-constructed boxes, no model needed	Detector smoke tests — confirm the actual model loads and runs
Compliance evaluation — feed synthetic assignment dicts	Full pipeline integration test — confirm real components connect correctly
Service orchestration order — mock detector/tracker to return canned outputs and assert call order	

MENTOR NOTE

Keep the mocked, synthetic-box tests as your primary safety net — they run in milliseconds and don't need a GPU or model weights. Real-model tests are valuable but slower and slightly non-deterministic; run them separately (e.g. marked with pytest markers) so your fast test suite stays fast.

Build

```
# tests/test_detection.py — example shape, not full implementation
import numpy as np

def test_zero_overlap_gives_zero_iob():
    ...

def test_full_containment_gives_iob_near_one():
    ...

def test_evaluate_compliance_flags_missing_vest():
    ...

def test_detector_handles_empty_frame():
    frame = np.zeros((640, 640, 3), dtype=np.uint8)
    ...
```

Files touched this phase

```
tests/test_detection.py
tests/test_violation.py
tests/test_tracker.py
```

Run

```
pytest tests/ -v
```

Checkpoint — Move to Phase 10 when...

- ☐ Every function in `violation.py` has at least one passing unit test
- ☐ The detector's empty-frame case is tested and passes
- ☐ At least one full-pipeline integration test passes on a real clip
- ☐ `pytest tests/ -v` runs green

Phase 10 — Dockerization & Deployment

Goal: package the whole modular monolith so it runs identically anywhere.

Learn First — Docker Basics

Term	Meaning
Image	A frozen snapshot of an environment (OS layer + dependencies + your code) — a blueprint
Container	A running instance of an image — the actual live process
Dockerfile	The recipe describing how to build the image, step by step
Build vs Run	Build creates the image once; run starts (potentially many) containers from it
Ports	Explicit mapping needed to reach a service inside the container from your host machine

The README's Dockerfile, Line by Line

```
FROM python:3.10-slim
```

Start from a minimal Python 3.10 base image — smaller than the full `python:3.10` image, faster to build and deploy.

```
RUN apt-get update && apt-get install -y \
    libgl1-mesa-glx \
    libglib2.0-0 \
    ffmpeg \
    && rm -rf /var/lib/apt/lists/*
```

OpenCV's Python wheel depends on system-level graphics libraries that the slim base image doesn't include by default — this is the single most common Docker build failure for CV projects if omitted. `ffmpeg` is required for video codec support. Cleaning the apt cache in the same layer keeps the image smaller.

```
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

Copying `requirements.txt` and installing dependencies *before* copying the rest of the code is a deliberate layer-caching optimization: Docker only re-runs the (slow) pip install step when dependencies actually change, not on every code edit.

```
EXPOSE 8501

ENTRYPOINT ["streamlit", "run", "src/dashboard/app.py", "--server.port=8501", "--server.address=0.0.0.0"]
```

`EXPOSE` documents the intended port; `--server.address=0.0.0.0` is required for Streamlit to be reachable from outside the container — the default `localhost` binding would only be reachable from inside the container itself.

Build and Run

```
docker build -t ppe-detection-system .
docker run -p 8501:8501 ppe-detection-system
```

GPU Docker Considerations

> ***THE README DOES NOT SPECIFY THIS***
>
> *GPU support inside Docker. The base Dockerfile above runs CPU-only inference. If you need GPU acceleration inside a container, you need the NVIDIA Container Toolkit on the host, a CUDA-enabled base image (e.g. `nvidia/cuda:...-runtime` or a PyTorch CUDA image), and to run with `docker run --gpus all`. This is a meaningfully different, more complex setup — treat it as a separate task from the base CPU Dockerfile, and only pursue it once the CPU container works end-to-end.*

Test

1. Build the image locally and confirm it completes without error
2. Run the container and open `http://localhost:8501` — the dashboard should load
3. Upload a test image/video inside the containerized dashboard and confirm inference works

Common Problems

Symptom	Likely cause
`ImportError: libGL.so.1` at container startup	Missing `libgl1-mesa-glx` system dependency
Dashboard unreachable from host browser	Missing `-p 8501:8501` port mapping, or Streamlit bound to `localhost` instead of `0.0.0.0`
Image is enormous (multiple GB)	Expected for a PyTorch + CUDA setup; for CPU-only, verify you're not accidentally pulling a CUDA-enabled torch wheel

Checkpoint — Move to Production Engineering when...

- ☐ The container builds cleanly from a fresh clone
- ☐ The dashboard is reachable and functional from the host browser
- ☐ You understand why each Dockerfile line exists, not just that it works

Production Engineering

What turns a working prototype into something you'd trust to run unattended.

Why This Matters

This is what separates "it worked when I demoed it" from a system a safety officer could actually rely on for a shift. None of this is optional if you want the project to read as production-minded in a portfolio review.

Area	What to do	Why
Configuration management	All thresholds/paths in `Settings` (Phase 1), overridable via environment variables	Deploy to a new environment without editing code
Logging	Structured, leveled logs at every pipeline stage boundary	Diagnose failures in a deployed system you can't step through with a debugger
Error handling	Catch and log expected failure modes (corrupt frame, dropped RTSP connection) instead of letting the process die	A crashed process means zero monitoring until someone notices and restarts it
Directory management	Ensure output directories exist at startup; don't assume they do	Prevents a first-run crash on a fresh deployment
Environment variables	Secrets and environment-specific config (model path, alert webhook URLs) via env vars, not hardcoded	Standard practice; also required for Docker deployments
Model versioning	Name weight files with a version/date (`best_v2_2026-08.pt`) rather than always overwriting `best.pt`	Lets you roll back if a retrained model regresses
Reproducibility	Pin dependency versions, log the exact training config used per model version	You should be able to explain exactly how any deployed model was produced
Performance monitoring	Log per-frame processing latency and FPS over time	Detect silent performance degradation before it becomes visible downtime
Resource management	Release camera/video handles properly on shutdown; avoid memory growth over long-running sessions	A dashboard meant to run for an 8-hour shift cannot leak memory

Performance Optimization

Only after correctness — measure before you optimize.

COMMON MISTAKE

Reaching for ONNX/TensorRT export before you've profiled where time is actually being spent. If your bottleneck is video decoding or the association loop in Python, optimizing the model's inference speed alone won't move your end-to-end FPS much.

What to Measure First

Metric	How to measure
FPS (end-to-end)	Frames processed ÷ wall-clock seconds, measured across the full `process_frame` call
Per-stage latency	Time detector, tracker, association, and annotation separately to find the actual bottleneck
GPU utilization	`nvidia-smi` while running — low utilization suggests you're bottlenecked elsewhere (I/O, Python overhead), not the model
CPU bottlenecks	Frame decoding, drawing operations, and Python-level loops in the association logic are common culprits

Optimization Levers, in Rough Order of Effort

Lever	Effect	Tradeoff
Lower input resolution	Faster inference	May hurt small-object (helmet) detection accuracy
Smaller model (YOLOv8n vs s)	Faster inference	Lower accuracy ceiling
Batch inference	Better GPU utilization if processing multiple streams	Adds latency per individual frame; only helps multi-camera setups
Reduce annotation overhead	Skip drawing traces/labels when not being viewed live	Less visual feedback during headless/batch processing, which is fine for that use case
ONNX export	Faster inference via optimized runtime, framework-agnostic	Extra export/validation step; some Ultralytics features may not carry over 1:1
TensorRT	Significant NVIDIA-GPU-specific speedup	NVIDIA-only, adds a build step tied to the specific GPU architecture
OpenVINO	Speedup on Intel CPUs/iGPUs	Useful specifically for CPU or Intel edge deployment, not GPU servers

When NOT to Optimize

MENTOR NOTE

If your current FPS already comfortably exceeds your camera's input frame rate, further optimization has zero practical benefit — you're already real-time. Spend that time instead on model accuracy, association robustness, or product features. Optimization work should be triggered by a measured, specific shortfall, not by a general feeling that things could be faster.

Checkpoint

- ☐ You have measured, not guessed, your current FPS and per-stage latency
- ☐ You can state which stage is actually the bottleneck
- ☐ Any optimization applied was chosen because it targets that specific bottleneck

Advanced Product Features

Only after the core MVP works end-to-end. Do not implement all of these at once.

COMMON MISTAKE

Treating this list as a checklist to complete before "the project is done." These are optional extensions that demonstrate product thinking — pick one or two that best complement your timeline and portfolio goals, not all six.

P1 Dynamic Geofencing & Danger Zones

Business value: Not all zones require the same PPE rules — a walkway may need only a vest, a crane zone may require a hard-hat and be fully restricted. Geofencing lets safety managers enforce zone-specific rules.

Technical complexity: Moderate — requires a polygon-drawing UI on the dashboard and a point-in-polygon check per worker position.

Prerequisites: Stable worker tracking (Phase 4b) and a working compliance engine (Phase 4c/4d).

Architecture impact: Adds a new zone-rules concept to the violation engine; the dashboard needs an interactive canvas/drawing widget.

Implementation order: After core compliance works reliably, before alerting (zone context makes alerts more meaningful).

P1 Instant Webhook / Telegram / Slack Alerts

Business value: Turns a passive dashboard into an active notification system — a safety officer doesn't have to be watching the screen.

Technical complexity: Low-to-moderate — mostly integration work (Telegram Bot API, Slack webhooks) plus rate-limiting so repeated frames of the same ongoing violation don't spam the channel.

Prerequisites: Snapshot saving from Phase 6, and a per-worker violation-state debounce so alerts fire on state *transitions*, not every frame.

Architecture impact: A new notification component subscribed to violation events from the service layer — keep it decoupled so it can be disabled without touching detection code.

Implementation order: After the core pipeline and snapshot saving are solid; this is a strong second feature to add.

P2 Automated PDF Safety Audit Reports

Business value: Turns raw logs into something a safety manager can print and file — daily compliance %, violation counts, historical trend.

Technical complexity: Low — mostly aggregation of already-logged data plus a PDF generation library.

Prerequisites: Metrics tracked over time (Phase 6), ideally persisted to a small database or structured log rather than only in-memory.

Architecture impact: Requires some form of persistent storage for historical stats beyond a single session — a meaningful new piece of infrastructure.

Implementation order: After you have persistent metric storage; a good showcase feature once the core system is stable.

P2 Audio / Visual Siren Triggers

Business value: Immediate, unmissable feedback for sustained violations, useful in a live monitoring room.

Technical complexity: Low technically, but requires careful UX design — needs a sustained-duration threshold to avoid constant false alarms on momentary detection flicker.

Prerequisites: Stable per-worker violation state over consecutive frames (depends on tracking quality from Phase 4b).

Architecture impact: Minor — a dashboard-side audio trigger keyed to a duration-based rule in the service layer.

Implementation order: Nice-to-have polish once core compliance detection is trustworthy — a jittery detector will produce an annoying, untrustworthy siren.

P2 Edge Inference Optimization (ONNX / TensorRT / OpenVINO)

Business value: Enables deployment on cheaper edge hardware (Jetson, Intel NUC) instead of requiring a full GPU server per camera.

Technical complexity: Moderate-to-high, hardware-specific, covered in Performance Optimization above.

Prerequisites: A finalized, evaluated model — re-exporting every time you retrain is wasted effort.

Architecture impact: The detector wrapper (Phase 4a) needs to support swapping inference backends behind the same interface.

Implementation order: Late — only once model accuracy and the rest of the pipeline are finalized.

P3 Multi-Camera & RTSP Stream Switcher

Business value: Realistic for actual industrial deployment, where one site has many cameras.

Technical complexity: High — concurrent stream handling, per-camera state management, and UI for switching between feeds without losing state.

Prerequisites: Rock-solid single-stream RTSP handling (Phase 5b) first; multi-camera multiplies any existing instability.

Architecture impact: Significant — the service layer needs to manage multiple concurrent pipelines, likely requiring async/threading changes.

Implementation order: Last. This is genuinely a scope expansion, not a polish feature — treat it as a "phase 2 of the whole project" if you pursue it.

Learning Roadmap

What Should I Learn While Building This Project? — Just-In-Time, not upfront.

MENTOR NOTE

Do not go complete a 40-hour "Deep Learning Specialization" before writing a single line of this project. Learn each concept right before the phase that needs it — you'll retain it far better when you immediately apply it, and you avoid the trap of endless preparatory learning that never turns into a shipped project.

Concept	Learn Before	Why
Python OOP (classes, composition)	Detector (Phase 4a)	Every core module is a class with a clear interface — architecture depends on this
NumPy array basics	Video Processor (Phase 5b)	Frames are NumPy arrays; slicing, shapes, and dtypes come up constantly
Object detection fundamentals	Training (Phase 3)	You cannot tune or debug a model you don't conceptually understand
IoU / IoB geometry	Violation (Phase 4c)	The mathematical core of the association logic
Multi-object tracking basics	Tracking (Phase 4b)	Needed to reason about ID stability and occlusion behavior
Streamlit fundamentals	Dashboard (Phase 7)	Its rerun execution model is unusual if you've only used Flask/Django before
Pytest basics	Testing (Phase 9)	Fixtures, parametrization, and assertion style for a real test suite
Docker fundamentals	Deployment (Phase 10)	Images vs. containers, layer caching, before touching the Dockerfile

Debugging Playbook

If Something Breaks, Do This.

Environment Error

Symptoms: ImportError, ModuleNotFoundError, version conflicts at startup.
Likely causes: Virtual environment not activated; dependency installed in the wrong environment; version mismatch between packages.
Inspect: `pip list` inside the active venv; confirm you're using the venv's Python, not system Python.
Minimal reproduction: A one-line script that only does the failing import.
Fix strategy: Reinstall in a clean venv if conflicts are tangled; pin versions once stable.
Prevention: Always activate the venv before installing or running anything; keep `requirements.txt` current.

Import Error (project-internal)

Symptoms: `ModuleNotFoundError: No module named 'src'` or similar, when running scripts from different directories.
Likely causes: Running a script from the wrong working directory; missing `__init__.py` files; Python path not including the project root.
Inspect: Your current working directory and `sys.path`.
Fix strategy: Run scripts consistently from the project root, or install the project in editable mode (`pip install -e .`) once you have a `setup.py` / `pyproject.toml`.
Prevention: Establish one consistent way of running scripts early and stick to it.

Dataset Error

Symptoms: Training crashes with a label/image mismatch, or trains but with nonsensical results.
Likely causes: Corrupt images, mismatched image/label filenames, wrong class indices in `data.yaml`.
Inspect: Overlay labels visually on a sample (Phase 2b); verify filename correspondence between `images/` and `labels/` folders.
Fix strategy: Re-run your EDA notebook's corrupt/mismatch checks against the full dataset, not just a sample.
Prevention: Never skip Phase 2b, even when in a hurry to start training.

YOLO Training Error

Symptoms: Training crashes mid-epoch, or loss is NaN.
Likely causes: Corrupted image in the dataset, a learning rate too high, or a malformed label file.
Inspect: Which epoch/batch it failed on; Ultralytics usually logs the offending file.
Fix strategy: Isolate and remove/fix the specific offending file rather than restarting with different hyperparameters blindly.
Prevention: A thorough Phase 2b EDA pass catches most of these before training even starts.

CUDA / GPU Error

Symptoms: `CUDA out of memory`, or `torch.cuda.is_available()` returns False unexpectedly.
Likely causes: Batch/image size too large for available VRAM; CPU-only PyTorch build installed; another process holding GPU memory.
Inspect: `nvidia-smi` to see current GPU memory usage and running processes.
Fix strategy: Reduce batch size first; verify your PyTorch install matches your CUDA driver version.
Prevention: Confirm GPU availability in your Phase 1 smoke test before relying on it for training.

Detection Error

Symptoms: `detect()` crashes, or returns clearly wrong boxes.
Likely causes: Wrong frame color format (RGB vs BGR mismatch), wrong input shape, or model path pointing at the wrong weights file.
Inspect: Print `frame.shape` and `frame.dtype` right before calling detect(); visually save the frame you're passing in.
Fix strategy: Verify against a known-good single test image before debugging the full pipeline.

Prevention: Keep Phase 4a's tests (valid image, empty image) running as a fast sanity check whenever you change detector code.

Tracking Error

Symptoms: IDs churn constantly, or never assign at all.
Likely causes: Feeding unfiltered detections (including PPE boxes) into the tracker instead of person-only detections.
Inspect: Print the class filter right before calling `tracker.update()`.
Fix strategy: Confirm you're filtering to `person` class before tracking, per Phase 4b.
Prevention: Keep the tracker test clip from Phase 4b as a standing regression check.

Association Error

Symptoms: PPE consistently assigned to the wrong worker, or never assigned at all.
Likely causes: IoB threshold too strict/loose; region slicing (head/torso %) miscalculated; comparing against the wrong candidate set.
Inspect: Print the computed IoB values for every person/PPE pair in a single test frame.
Fix strategy: Go back to the unit tests in Phase 9 with hand-computed boxes before touching real frames.
Prevention: Never modify `violation.py` without running its unit test suite immediately after.

Video Error

Symptoms: `cap.read()` always returns False, or frames are corrupted/green/garbled.
Likely causes: Wrong codec, unsupported container format, or (for RTSP) a dropped/unstable network connection.
Inspect: Try the exact same source path directly in VLC or another player to isolate whether it's an OpenCV issue or a source issue.
Fix strategy: Confirm the source plays correctly outside your code first, then narrow down OpenCV-specific handling.
Prevention: Build and test against MP4 before webcam before RTSP, per Phase 5b.

Streamlit Error

Symptoms: State resets unexpectedly, UI freezes, or widgets don't update.
Likely causes: Misunderstanding Streamlit's rerun-on-interaction model; state not stored in `st.session_state`.
Inspect: Add explicit print/log statements to see exactly when your script re-executes.
Fix strategy: Move any state that must survive a rerun into `st.session_state` explicitly.
Prevention: Read Streamlit's execution model documentation before building V3 (the live-loop version).

Docker Error

Symptoms: Build fails, or container starts but the dashboard is unreachable.
Likely causes: Missing system dependencies (see Phase 10), wrong port mapping, Streamlit bound to localhost instead of 0.0.0.0.
Inspect: `docker logs <container_id>` for the actual startup error.
Fix strategy: Reproduce the failure by running the same commands manually inside an interactive container (`docker run -it ... bash`).
Prevention: Test the container locally after every Dockerfile change, not just at the end of the project.

Git Strategy

Small, meaningful commits tell the story of how the system was built.

Suggested Commit Sequence

```
chore: initialize project structure
feat: add dataset exploration
feat: add YOLO training pipeline
feat: add PPE detector
feat: add worker tracking
feat: add compliance engine
feat: add frame processing
feat: add detection service
feat: add Streamlit dashboard
test: add compliance tests
chore: dockerize application
```

Why Small, Meaningful Commits Matter

- **Debuggability:** `git bisect` only works well if each commit represents one coherent change
- **Portfolio signal:** A reviewer skimming your commit history should be able to reconstruct your engineering process — this is itself evidence of disciplined work
- **Safe rollback:** If a change to the association logic breaks something, you want to revert exactly that change, not an entangled mix of five unrelated edits
- **Review clarity:** Each commit should be reviewable on its own — "what changed and why" should be answerable from the diff and message alone

Use the conventional-commit prefixes above (`feat` , `test` , `chore` , `fix`) consistently — it costs nothing and makes your history scannable.

Definition of Done

Check every item before calling the project complete.

Dataset

- ☐ Train/val/test split with no leakage
- ☐ Class distribution documented
- ☐ Corrupt/duplicate images removed
- ☐ Approach A vs. B decision (Phase 2) documented

Model

- ☐ Baseline trained and evaluated
- ☐ Per-class metrics recorded in the benchmark table
- ☐ Confusion matrix reviewed, confused classes documented
- ☐ Known weaknesses written down explicitly

Detector

- ☐ `PPEDetector.detect()` works on real and empty frames
- ☐ Model loaded once, not per-frame
- ☐ Baseline latency measured

Tracker

- ☐ Stable IDs on a single-worker clip
- ☐ Multi-worker ID assignment verified
- ☐ ID-swap/occlusion behavior observed and documented

Compliance

- ☐ All violation.py functions unit tested
- ☐ Full edge-case matrix (Phase 4c) covered
- ☐ UNKNOWN state decision documented

Video

- ☐ MP4 works
- ☐ Webcam works
- ☐ RTSP attempted, with reconnection handling

Service

- ☐ `process_frame()` runs end-to-end on a full clip
- ☐ Snapshot throttling works correctly
- ☐ Running metrics verified against manual expectation

Dashboard

- ☐ Versions V1–V5 (at minimum) working
- ☐ Metrics cards reflect live state
- ☐ Violation gallery displays saved snapshots

CLI

- [ ] All three subcommands work
- [ ] Invalid input produces a clear error, not a crash

Tests

- [ ] `pytest tests/ -v` passes fully
- [ ] Geometry, compliance, detector, tracker, and pipeline all covered

Docker

- [ ] Image builds cleanly from a fresh clone
- [ ] Container runs and dashboard is reachable

Documentation

- [ ] README reflects the actual, current state of the system
- [ ] Any deviations from the original spec explained
- [ ] Setup/run instructions verified by literally following them yourself on a clean checkout

Suggested Development Timeline

A realistic pace — treat these as guardrails, not a strict deadline.

Week	Focus	Notes
Week 1	Dataset (Phase 2, 2b) + YOLO Training (Phase 3, 3b)	Often takes longer than expected — dataset quality issues surface here and are worth the extra time to fix properly
Week 2	Detector (4a) + Tracking (4b) + Compliance (4c, 4d)	The association logic (4c) is the intellectual core of the project — don't rush it to stay "on schedule"
Week 3	Video Processing (5b) + Service (6) + Dashboard (7)	Build the dashboard incrementally (V1–V7); resist building it all in one sitting
Week 4	Testing (9) + Docker (10) + product polish	Testing should really be happening continuously since Phase 4c, not compressed into one week — treat this week as catch-up and hardening

MENTOR NOTE

If you're building this around a job or coursework, this maps more realistically to 4–8 weeks at a slower pace. The phase that most often runs long is dataset work (Week 1) — budget slack there rather than compressing Phase 4c (association logic) to make up time, since that's where the real engineering learning happens.

Final Portfolio / Product Standard

What separates a simple YOLO project from an AI Engineering product.

Dimension	Simple YOLO project	AI Engineering product (this guide's target)
Architecture	One script that runs inference	Layered modular monolith with clear boundaries
Model quality	Trained once, mAP reported without scrutiny	Per-class metrics reviewed, known weaknesses documented
Business logic	None — detection output is the end product	Spatial association, compliance rules, explicit state modeling
Tracking	Absent, or naive frame-to-frame matching	Proper multi-object tracker with documented limitations
Testing	None, or manual only	Automated unit and integration tests, especially for business logic
Logging	`print()` statements	Structured, leveled, rotated logs
Configuration	Hardcoded paths and thresholds	Centralized, typed, environment-overridable settings
UI	None, or a bare script	Purpose-built dashboard for the actual end user (safety officer)
Deployment	"Runs on my machine"	Containerized, reproducible
Documentation	Auto-generated or absent	Explains architecture decisions and known limitations honestly
Performance	Never measured	FPS/latency measured, optimization decisions justified by data
Product thinking	"I trained a model"	"I identified a real workflow, built the pipeline to support it, and know its limits"

What to Demonstrate in a Final Interview or Portfolio Presentation

- Walk through the architecture diagram from memory and explain *why* each layer exists
- Explain the IoB association decision and why it beats naive IoU for this problem
- State your model's known weaknesses honestly, with numbers, rather than claiming it's perfect
- Explain ByteTrack's re-identification limitation and how it would affect a real deployment
- Show your test suite running green, and explain what it does and does not cover
- Explain one engineering tradeoff you made under time or data constraints, and what you'd do differently with more time

Final Demo Scenario

What a complete, working demo looks like.

```
Safety Manager
        |
        v
Opens Dashboard
        |
        v
Selects CCTV / video source
        |
        v
System detects workers
        |
        v
Tracks workers (stable IDs)
        |
        v
Checks PPE (helmet, vest) per worker
        |
        v
Detects violation -> red box + status label
        |
        v
Captures evidence snapshot
        |
        v
Updates dashboard KPIs live
        |
        v
Shows historical analytics
```

What Should Be Visible During the Demo

- Live annotated video with color-coded boxes (green = compliant, red = violation, yellow = PPE items)
- Worker IDs remaining stable as people move through frame
- KPI cards updating in real time as workers enter/leave and compliance changes
- A violation snapshot appearing in the gallery the moment a violation is detected
- At least one analytics chart (compliance rate over time, or violation type breakdown) with real accumulated data

Final Architecture — Layered View

Every file mapped to its layer.

```
Presentation Layer
        |
        v
Application Service Layer
        |
        v
Domain / CV Layer
        |
        v
Infrastructure / I/O
        |
        v
Model / Data
```

Layer	Files
Presentation	`src/dashboard/app.py`, `main.py` (CLI)
Application Service	`src/services/detection_service.py`
Domain / CV	`src/detection/detector.py`, `src/detection/tracker.py`, `src/detection/violation.py`, `src/processing/frame_processor.py`
Infrastructure / I/O	`src/processing/video_processor.py`, `src/core/logger.py`
Model / Data	`models/best.pt`, `data/`, `training/train.py`, `training/evaluate.py`, `src/core/config.py`

Engineering Rules to Live By

Reinforced throughout this guide — repeated here as a single reference.

- ☐ Do not over-engineer early
- ☐ Do not build the UI before the core pipeline works
- ☐ Do not blindly copy code you don't understand
- ☐ Do not tune the model before checking the dataset
- ☐ Do not optimize before measuring
- ☐ Do not use microservices unnecessarily
- ☐ Keep responsibilities separated between layers
- ☐ Test business logic independently of the model and the UI
- ☐ Build incrementally, one working stage at a time
- ☐ Make decisions based on measurements, not assumptions
- ☐ Understand every module you implement — you should be able to explain it without notes

CLOSING MENTOR NOTE

You now have the complete roadmap. Come back to this guide every time you finish a phase — re-read the checkpoint, confirm it honestly, and only then move forward. The goal was never to get to Phase 10 fast; it was to understand, at every layer, why the system is built the way it is. That understanding is what will make this project defensible in an interview and genuinely useful as a foundation for whatever you build next.